

三分查找

求单峰函数极值

北京大学 王欣灏



问题回顾：求方程的根



求方程 $4x^2-8x+1=0$ 在 $[0, 1]$ 间的**实根**，结果保留四个小数位。

设 $f(x)=4x^2+8x-1$ 有： $f(0)=1$ $f(1)=-3$

我们知道，若区间 (a,b) 内有根，则必有 $f(a)*f(b)<0$

我们可以用二分法来讨论它的根

步骤0. 设 $L=a, R=b$, 反复执行下列步骤：

步骤1. 设 $mid=(L+R)/2$;

步骤2. 若 $f(mid)=0$, 则: mid 为根，结束；

步骤3. 若 $f(L)*f(mid)<0$ 则:

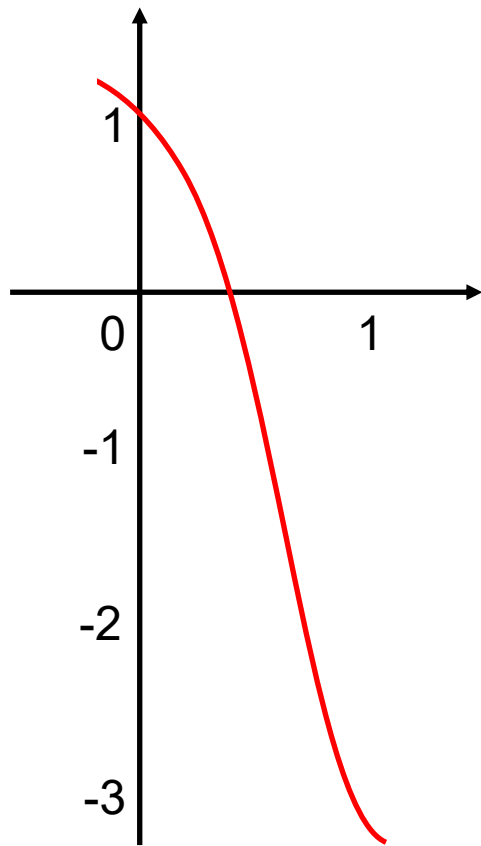
根在区间 (L, mid) 中， $R=mid$, 重复执行步骤1。

若 $f(L)*f(mid)>0$, 则:

必然有 $f(R)*f(mid)<0$, 根在区间 (mid, R) 中，
 $L=mid$, 重复执行步骤1；

但是，因为这是一个实数方程，很难出现 $f(x)=0$ 的情况，所以，**只要一个实数足够接近0, 我们就认为它与0相等。**

比如：if (**fabs**($0-f(x)$) ≤ 0.00001) 我们认为 $f(x)$ 足够接近0了.....



【一元三次方程求解 NKOI 2000】

有形如： $ax^3+bx^2+cx+d=0$ 这样的三个一元三次方程。
给出该方程中各项的系数(a, b, c, d均为实数)，并约定该
方程存在三个不同实根(根的范围在-10000至10000之间)，
且根与根之差的绝对值 ≥ 1 。

要求由小到大依次在同一行输出这三个实根(根与根之间留
有空白)，并精确到小数点后4位。



分析

- A.当已知区间 (a,b) 内有一个根时;
- 用二分法求根, 若区间 (a,b) 内有根, 则必有 $f(a)*f(b)<0$ 。重复执行如下的过程:
 - ①、若 $abs(a-b)<0.00001$ 或 $f((a+b)/2)=0$, 则可确定根为 $(a+b)/2$ 并退出过程;
 - ②、若 $f(a)*f((a+b)/2)<0$, 则由题目给出的定理可知根在区间 $(a,(a+b)/2)$ 中, 故对区间重复该过程;
 - ③、若 $f(a)*f((a+b)/2)>0$, 则必然有 $f((a+b)/2)*f(b)<0$, 根在 $((a+b)/2,b)$ 中, 对此区间重复该过程。
- 执行完毕, 就可以得到精确到0.0001的根。



分析

- 求方程的所有三个实根
- 所有的根的范围都在-10000至10000之间，且根与根之差的绝对值 ≥ 1 。
- 因此可知：在 $[-10000, -9999]$ 、 $[-9999, -9998]$ 、.....、 $[9999, 10000]$ 这20000个区间内，每个区间内至多只能有一个根。
- 只有当 $f(a)=0$ 或 $f(a)*f(a+1)<0$ 时，方程在此区间内才有解。若 $f(a)=0$ ，解即为 a ；若 $f(a)*f(a+1)<0$ ，则可以利用A中所述的二分法迅速找出解。如此可求出方程的所有解。



【参考代码】

```
double F(double x)
```

```
{  
    return a*x*x*x+b*x*x+c*x+d;  
}
```

```
void Solve(double L,double R)
```

```
{  
    double mid,y;  
    while(fabs(R-L)>=0.000001) //根据题意，设置足够高的精度  
    {  
        mid=(R+L)/2;  
        y=F(mid);  
        if(fabs(y)<=0.000001){ printf("%.4lf ",mid); return; } //F(mid)=0  
        if(F(L)*y<0)R=mid; //根在L到Mid之间  
        else L=mid; //根在Mid到R之间  
    }  
    printf("%.4lf ",mid);  
}
```



【参考代码 递归版】

```
void Solve(double L,double R)
{
    double mid,y0,y1,y2;
    mid=(L+R)/2;
    y1=F(L);
    y2=F(R);
    y0=F(mid);
    if(R-L<0.000001 && y1*y2<0)
    {
        printf("%.4lf ",mid);
        return;
    }
    if(y1*y0<0)Solve(L,mid);
    if(y0*y2<0)Solve(mid,R);
}
```



```
int main()
```

```
{
```

```
.....
```

```
if(F(-10000)==0)
```

\\讨论左边界-10000

```
{
```

```
    printf("%.4lf ",double(-10000));
```

\\double(-10000),将整数-10000强制转换为实数

```
    cnt++;
```

```
}
```

```
for(int i=-10000;i<10000;i++)
```

\\依次讨论-10000到9999的每一个长度为1的区间

```
{
```

```
    if(cnt==3)break;
```

```
    y1=F(double(i));
```

\\讨论[i,i+1]区间是否有解

```
    y2=F(double(i+1));
```

```
    if(y1*y2<0)
```

\\f(i)*f(i+1)<0,则区间[i,i+1]内一定有一个解

```
{
```

```
        Solve(double(i),double(i+1));
```

```
        cnt++;
```

```
}
```

```
else if(y2==0)
```

\\f(i+1)==0,则i+1就是一个解

```
{
```

```
    printf("%.4lf ",double(i+1));
```

```
    cnt++;
```

```
}
```

```
}
```

```
.....
```

```
}
```



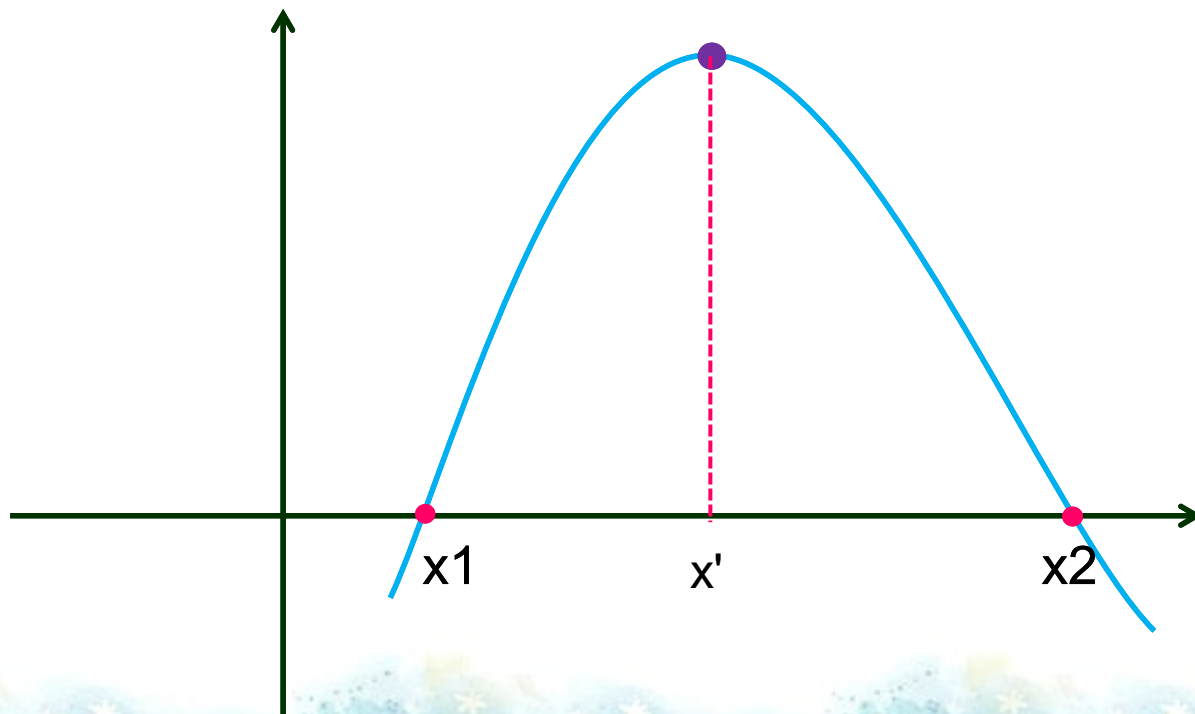
三分查找



【三分法】

二分查找所面向的搜索序列的要求是：具有**单调性**（不一定严格单调）；**没有单调性的序列不能使用二分查找。**

三分法所面向的搜索序列的要求是：**序列为一个凸性函数（单峰函数）**。
通俗来讲，就是该序列必须有一个最大值（或最小值），在最大值（最小值）的左侧序列，必须满足不严格单调递增（递减），右侧序列必须满足不严格单调递减（递增）。如下图，表示一个有最大值的凸性函数：



求一个 x' ，使得 $f(x')$ 的值最大，也就是位于紫色点。

思路：设 $L=x1, R=x2$ ，通过不断缩小 $[L, R]$ 的范围，无限逼近紫色点。



【三分法的过程】

步骤1.初始化: $L=x_1, R=x_2$;

步骤2.为将讨论区间分成三段, 我们需要产生两个分界点:

$$Lmid = L + (R-L)/3 = (2L+R)/3$$

将 $[L,R]$ 分成了三个区间

$$Rmid = R - (R-L)/3 = (2R+L)/3$$

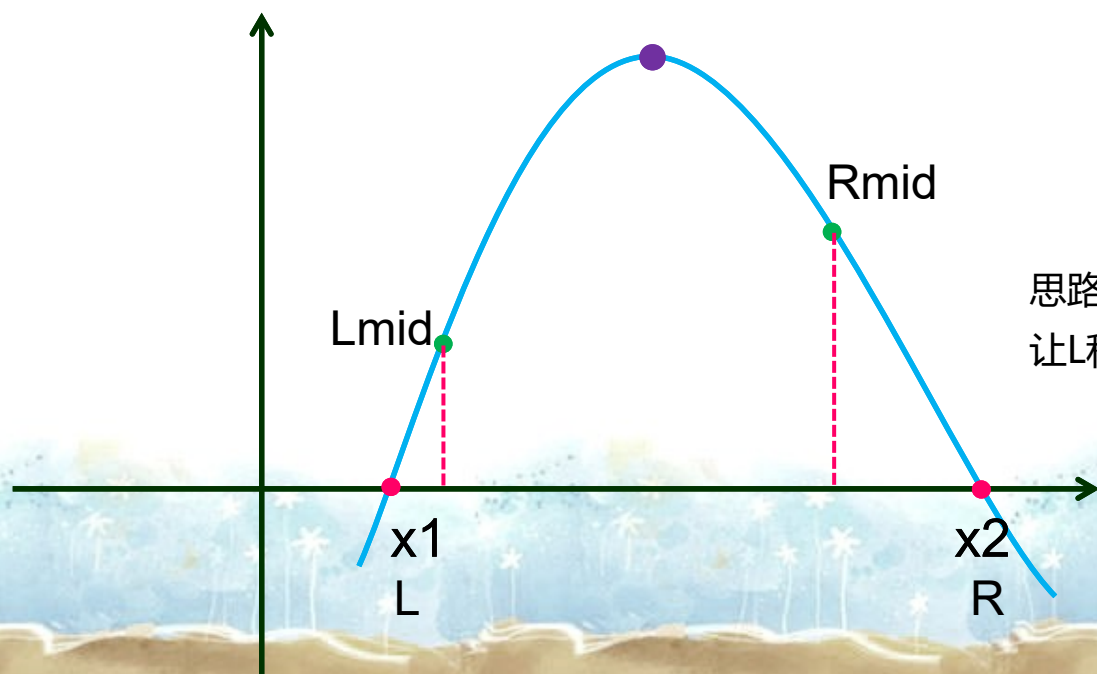
$[L, Lmid], [Lmid, Rmid], [Rmid, R]$

步骤3.讨论 $Lmid$ 与 $Rmid$ 谁更加接近极值

if($f(Rmid) \geq f(Lmid)$) $L=Lmid$; 再执行第2步。//相当于抛弃最左边的区间 $[L, Lmid]$

if($f(Rmid) < f(Lmid)$) $R=Rmid$; 再执行第2步。//相当于抛弃最右边的区间 $[Rmid, R]$

if($Rmid == Lmid$) return $(Lmid + Rmid)/2$; //相当于中间区间 $[Lmid, Rmid]$ 缩减"足够小"

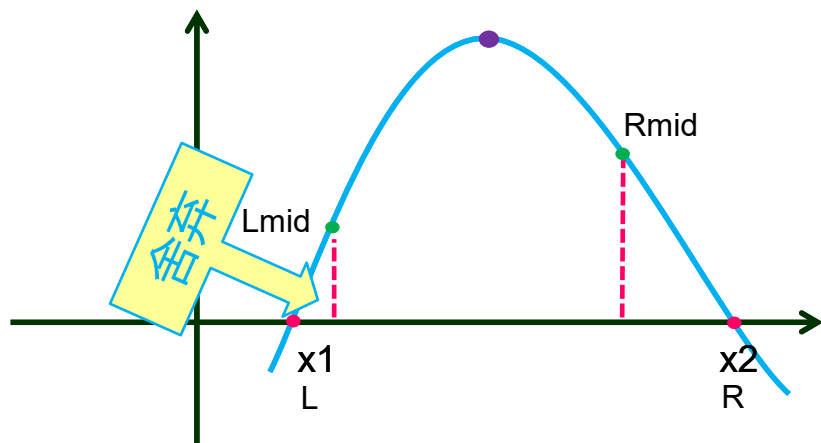


思路: 设 $L=x_1, R=x_2$, 通过不断缩小 $[L,R]$ 的范围, 让 L 和 R 去无限逼近紫色点。

【算法正确性】

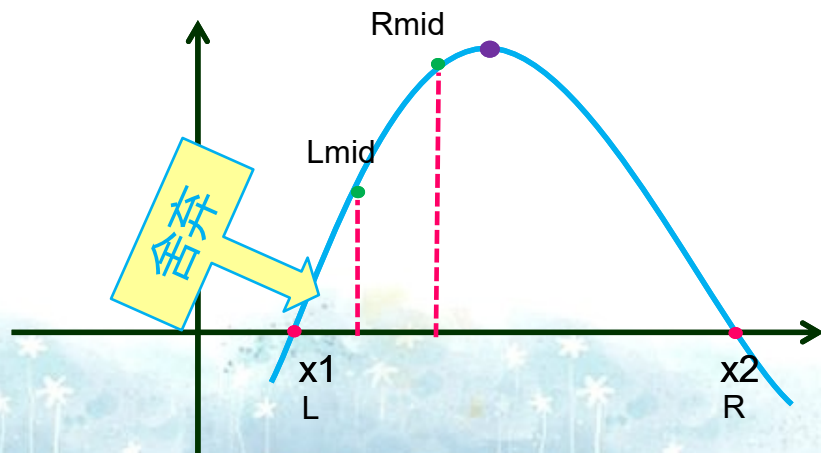
情况1: L_{mid} 与 R_{mid} 在最值的两侧。

由于最值在中间的一个区间 $[L_{mid}, R_{mid}]$ ，因此我们舍弃一个区间(比如下图的 $[L, L_{mid}]$)后，并不会影响到最值。



情况2: L_{mid} 与 R_{mid} 在最值的同一侧。

由于凸性函数在最值在任意一侧都具有单调性，因此， L_{mid} 与 R_{mid} 中，更大的那个数自然更为靠近最值。此时，远离最值的那个区间是不可能包含最值的，因此可以舍弃。



【代码模板】

```
double Three_Devide(double L,double R)
{
    double Lmid,Rmid;
    while(R-L>=0.00001) //当L与R非常接近, 当作L==R, 具体精确范围根据题目来设定
    {
        Lmid=L+(R-L)/3;
        Rmid=R-(R-L)/3;
        if(f(Lmid)<=f(Rmid))L=Lmid; //舍弃[L,Lmid]
        else R=Rmid; //舍弃[Rmid,R]
    }
    return (Lmid+Rmid)/2;
}
```

时间复杂度 $O(\log_{\frac{3}{2}}n)$



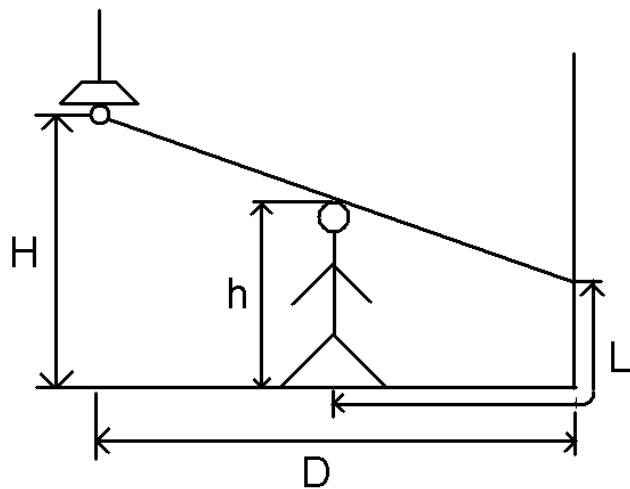
记住：

若我们能把某个问题抽象成一个在某段区间 $[a,b]$ 内呈上凸或下凹的图像造型，我们都可以考虑使用三分法来求解该问题在这段区间内的极值。



影子

- 科学馆机房门口有条狭窄的走廊，走廊尽头是一堵墙。走廊的顶部有一盏吊灯。何老板发现，在吊灯与墙之间行走时，人的影子的长度会发生变化，他想知道，影子最长是多长？现在他告诉你吊灯的高度 H 、何老板的身高 h 、吊灯与墙的水平距离 D ，请你帮他算出影子最长的长度值 L （具体情形如下图所示）。



影子

- 需要考虑哪些情况？
- 两种？一种？
- 如何构造函数表达式？
- 是否满足三分性质？



影子

- 第一种情况（墙上没有），影子长度随着何老板离灯的距离递增（你通过相似三角形列出数学公式发现是线性函数（1次）），然后最优解就直接是投影到墙根的位置。相似三角形比一下，
$$\text{Ans1} = (D \cdot h) / H;$$
- 第二种情况，投影到墙上。那么就有数学函数推导式：影子长度 = $H + D - (\text{何老板到墙的距离} + D \cdot (H - h) / \text{何老板到墙的距离})$;



影子

- 对于第二种情况
函数表达式形如 $len = ax + \frac{b}{x}$, $a > 0, b > 0$
根据数学知识知道在 $x = \sqrt{ab}$ 时 len 取最小值
- 故对于该题第二种情况
何老板到墙的距离 = $\sqrt{D(H-h)}$ 时取最大值
- 或者套用三分法求到函数最大值



影子

```
#include<stdio>
#include<cmath>
using namespace std;
int main()
{
    double H,h,D,L,x;
    scanf("%lf%lf%lf",&H,&h,&D);
    if (H-h>D)
        x=h;
    if (H-h<=D)
        x=H+D-2*(sqrt(D*H-D*h));
    printf("%.3lf\n",x);
    return 0;
}
```



影子

```
double H,h,D;
double f(double x){
    return H+D-(x+D*(H-h)/x);
}
int main(){
    double ans1,ans2;
    cin>>H>>h>>D;
    ans1=D*h/H;

    double l=0,r=h*D/H,Lmid,Rmid;
    while(r-l>=0.000001){
        Lmid=l+(r-l)/3;
        Rmid=r-(r-l)/3;
        if(f(Lmid)<=f(Rmid))l=Lmid;
        else r=Rmid;
    }
    printf("%.31f",max(f(l),ans1));
}
```



函数

- 给定两个下凸二次函数 $f(x), g(x)$ ，构造 $h(x) = \min\{f(x), g(x)\}$
- 求 $h(x)$ 在区间 $[0, 1000]$ 上的最小值



函数

- 一个二次函数如何计算？
- 计算对称轴 **or** 三分法
- 两个二次函数怎么办
- 法1：两条对称轴最多把区间分为三段，每一段均容易计算
- 法2：两条二次函数最多有两个交点，交点隔开的区间上，显然 $h(x)=f(x)$ 或 $h(x)=g(x)$ ，变为一条函数的情况，或者直接偷懒三分

